solution, whose quality increases with the quantity of time we allot for the algorithm to run.

When, instead, we have guarantees about the derivability and shape of the function (for instance if we are sure we have a bowl-shaped function, like for linear/logistic regression etc…), then we want to take advantage of the super-powers of gradient descent.

Are there cases where simulation annealing is best avoided?

Obviously, as we have discussed, if a problem doesn’t admit near-optimal solutions, but it rather demands the best possible one, then simulated annealing is not the best tool in our belt.

The shape of the cost function matters as well: when the cost function has narrow valleys, the algorithm will have a low probability of finding them – and if they hold the global minima, it’s unlikely simulated annealing will converge to a near-optimal solution.

Finally, as we will see in our examples, an important requirement is that the cost function should be easily calculated for a new candidate solution (possibly allowing to directly compute the delta based only on the difference with current solution): since this cost is computed at each iteration, a computationally-intensive cost function will slow down the optimization, forcing us to run the algorithm on fewer iterations.